

Day 24 – Annotations, Day 06

Practical exercise: a mini banking console application built with @Component, @Lazy, @Scope, @DependsOn, @PropertySource and @Value

This session brings together several annotations covered earlier into one working example: a small banking application that runs in the console. The user can deposit money, withdraw money, check the balance, and generate a report, all through a simple menu. The example is spread across five files, and each one is explained below in the order it runs.

1. BankServerStartup.java – Starting Up the Server

@Component marks this class as a Spring-managed bean. Since no custom name is given, Spring registers it with the default name **bankServerStartup** (the class name with its first letter changed to lowercase). Because it has no special scope or laziness setting, this bean is created as soon as the container starts up.

The constructor prints a series of startup messages in different colours using ANSI escape codes, then calls `Thread.sleep(3000)` to pause for three seconds. This simulates the time a real server would take to start up before it is ready to use.

```
package com.day24;

import org.springframework.stereotype.Component;

@Component
public class BankServerStartup {

    public BankServerStartup() {
        System.out.println("Server started");
        System.out.println("Security Verify");
        System.out.println("Database connected");
        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("All set up done");
    }
}
```

2. BankReportGeneration.java – A Lazy Bean

@Component("report") registers this class as a bean, but gives it the custom name **report** instead of the default name.

@Lazy tells Spring not to create this bean when the container starts. Instead, the bean is created only the first time it is actually requested with `getBean()`. This is useful for beans that are not needed by every user of the application – here, the report bean should only be created if the user chooses the "generate report" option.

The constructor prints messages that simulate processing report data. The `reportGenerated()` method prints a confirmation message once it is called.

```

package com.day24;

import org.springframework.context.annotation.Lazy;
import org.springframework.stereotype.Component;

@Component("report")
@Lazy
public class BankReportGeneration {

    public BankReportGeneration() {
        System.out.println("All format uploaded");
        System.out.println("All format converted into database");
        System.out.println("All the value is verified");
        System.out.println("All the environment is verified");
    }

    public void reportGenerated() {
        System.out.println("All customer data is stored!");
    }
}

```

3. BankOperation.java – The Core Banking Logic

This class carries several annotations together, each doing a different job:

- **@Component("forAllCustomer")** – registers this class as a bean with the custom name **forAllCustomer**.
- **@Scope("prototype")** – by default, a Spring bean is a singleton, meaning the same object is returned every time it is requested. Marking this bean as **prototype** changes that: every call to `getBean()` creates a brand new object instead.
- **@PropertySource("classpath:com/day24/application.properties")** – loads the given properties file into the Spring environment, so the keys inside it become available for injection with **@Value**.
- **@DependsOn("bankServerStartup")** – tells Spring that the `bankServerStartup` bean must be fully created before this bean is created. This makes sure the server start-up process finishes before any banking operation can begin.
- **@Value("\${...}")** – reads the value of the given property key from the environment (loaded through **@PropertySource**) and injects it directly into the field.

```

package com.day24;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.DependsOn;
import org.springframework.context.annotation.PropertySource;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component("forAllCustomer")
@Scope("prototype")
@PropertySource("classpath:com/day24/application.properties")
@DependsOn("bankServerStartup")

```

```

public class BankOperation {

    @Value("${Bank_UserName}")
    private String userName;

    @Value("${Bank_AccNumber}")
    private String accNumber;

    @Value("${Bank_currentBalance}")
    private long accountBalance;

    public BankOperation() {
        System.out.println("Bank operation all set to start");
    }

    public void deposit(long amount) {
        if (amount > 0) {
            accountBalance = accountBalance + amount;
            System.out.println("Amount added successfully");
        } else {
            System.err.println("Enter the valid amount");
        }
    }

    public void withdraw(long amount) {
        if (accountBalance >= amount && amount > 0) {
            accountBalance -= amount;
            System.out.println("Amount withdraw");
        } else {
            System.err.println("Invalid amount or Insufficient funds");
        }
    }

    public void showBalance() {
        System.out.println("current Balance is " + accountBalance);
    }
}

```

The three methods use the injected values: `deposit()` adds the given amount to the balance if it is a positive number, `withdraw()` subtracts the amount only if the balance is sufficient and the amount is positive, and `showBalance()` simply prints the current balance.

4. application.properties – The Property Values

This file holds the actual values that get injected into the fields of `BankOperation` through `@Value`. It is loaded because of the `@PropertySource` annotation on that class.

```

Bank_UserName=prashant
Bank_AccNumber=124124124
Bank_currentBalance=50

```

5. Main.java – Running the Application

`new AnnotationConfigApplicationContext("com.day24")` scans the `com.day24` package for annotated classes and builds the IoC container from them.

`context.getBean("forAllCustomer", BankOperation.class)` fetches the bean by its custom name and returns it as a `BankOperation` object.

A `Scanner` then reads the user's choice in a loop and shows a menu with five options:

- **1 – Deposit:** reads an amount and calls `deposit()`.
- **2 – Withdraw:** reads an amount and calls `withdraw()`.
- **3 – Check balance:** calls `showBalance()`.
- **4 – Generate report:** calls `context.getBean("report", BankReportGeneration.class)`. This is the first time the **report** bean is requested, so this is exactly when its `@Lazy` constructor actually runs.
- **5 – Exit:** prints a closing message and ends the program with `System.exit(0)`.

```
import java.util.Scanner;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {

    public static void main(String[] args) {
        System.out.println("Main method working");

        ApplicationContext context =
            new AnnotationConfigApplicationContext("com.day24");

        BankOperation bean =
            context.getBean("forAllCustomer", BankOperation.class);

        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.println("Enter your choice");
            System.out.println("Press 1 for deposit");
            System.out.println("Press 2 for withdraw");
            System.out.println("Press 3 for check balance");
            System.out.println("Press 4 for generate report");
            System.out.println("Press 5 for exit");

            int choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.println("Enter the amount for deposit");
                    long depositAmount = sc.nextLong();
                    bean.deposit(depositAmount);
                    break;
                case 2:
                    System.out.println("Enter the amount for withdraw");
```

```
long withdrawAmount = sc.nextLong();
bean.withdraw(withdrawAmount);
break;
case 3:
bean.showBalance();
break;
case 4:
BankReportGeneration bean2 =
context.getBean("report", BankReportGeneration.class);
bean2.reportGenerated();
break;
case 5:
System.out.println("Thanks for using this application. Namaste");
System.exit(0);
break;
default:
}
}
}
}
```

Key Takeaways

- **@Lazy** delays a bean's creation until it is actually requested with `getBean()` – here, the report bean is only built when option 4 is chosen.
- **@Scope("prototype")** makes Spring create a brand new object on every `getBean()` call, instead of reusing one shared instance.
- **@DependsOn** controls the order in which beans are created, without creating a direct dependency between their fields.
- **@PropertySource** loads a properties file into the Spring environment, and **@Value** reads individual keys from it into fields.